Fast Block Diffusion Training via Multi-Mask Context Sharing

Anonymous ACL submission

Abstract

Block Diffusion Language Models generate multiple blocks of text sequentially, while tokens within each block are denoised in parallel, enabling efficient parallel decoding. However, their training suffers from low computational utilization: each sample is duplicated into a clean context and a masked copy, yet only the masked portion contributes to the training loss, resulting in merely 50% effective computation. In this paper, we propose **Multi-Mask Training with context sharing** (**MMT**), a simple yet effective training paradigm that significantly improves computational efficiency through context sharing. Our key idea is to generate K different mask patterns for the same clean text and process them together in a single forward pass via a sparse attention mechanism, where each masked segment attends to the shared clean context while remaining isolated from other segments. Notably, our proposed approach yields better performance under equivalent training time constraints, despite relying on fewer unique training instances. We find $K{=}4$ to be optimal, which raises the effective utilization from 50% to 80% and yields a 1.6× training speedup with no degradation in perplexity (TODO) or downstream performance.

1 Introduction

The success of Large Language Models (Brown et al., 2020; Bommasani et al., 2021; Han et al., 2021; OpenAI, 2023) has been largely driven by autoregressive (AR) architectures. However, their sequential, token-by-token generation paradigm imposes significant latency bottlenecks during inference. Recently, Diffusion Language Models (Nie et al., 2025b; Khanna et al., 2025; Song et al., 2025; Google DeepMind, 2025) have emerged as promising alternatives. By training the model to generate tokens in parallel, these models offer a flexible trade-off between generation quality and inference speed.

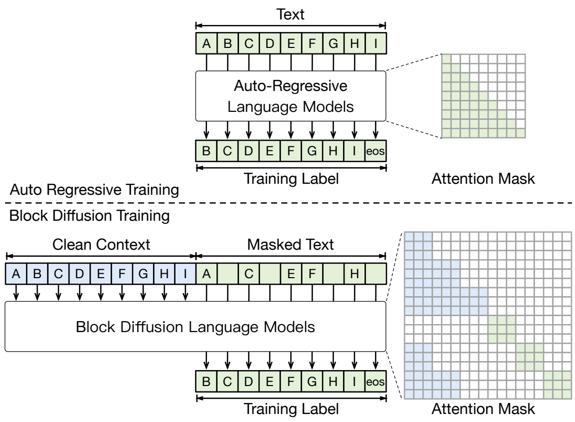

Figure 1: The training pattern of Auto-Regressive Language Models and Block Diffusion Language Models.

Block Diffusion Language Models (Arriola et al., 2025; Wu et al., 2025a; Cheng et al., 2025; Tian et al., 2025; Bie et al., 2025) further bridge the gap between autoregressive and diffusion paradigms. These models decompose the generation process into sequential blocks, where tokens within each block are generated via diffusion while blocks themselves are produced autoregressively. This hybrid formulation enables variable-length generation, supports KV caching across blocks, and allows parallel token sampling within each block, making it a compelling architecture for practical deployment.

Despite their promising inference properties, Block Diffusion Language Models suffer from a critical training inefficiency. As illustrated in Figure 1, each training sample is duplicated into two copies: one serves as the clean context, while the other is masked and used for loss computation. This duplication results in a mere 50% computational utilization, as half of the forward pass is spent on the context that does not contribute to the training signal. In contrast, autoregressive models achieve near-100% utilization since every token contributes to the training loss. Beyond the

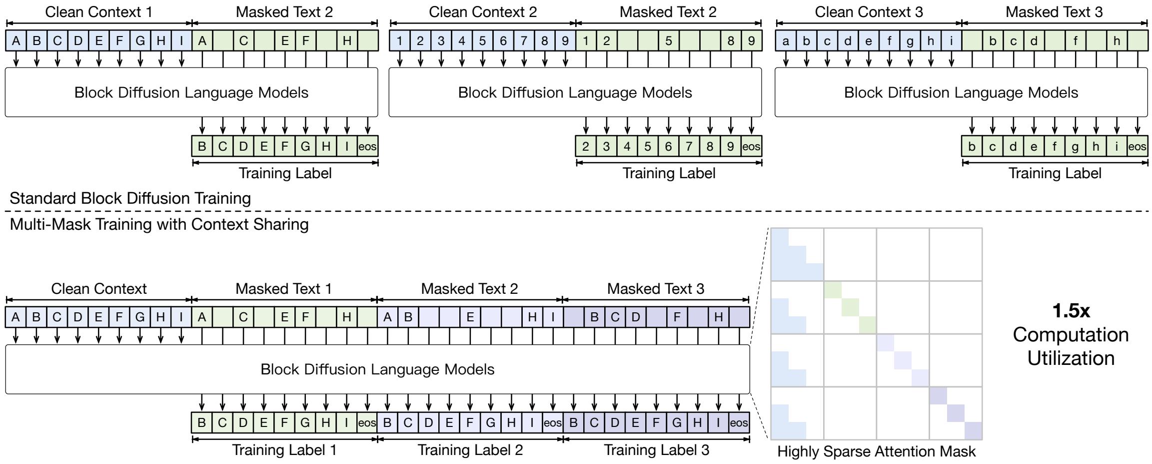

Figure 2: The training pattern of MMT compared to the original training pattern of Block Diffusion Language Models, when the number of different mask patterns K is 3. MMT train on less training instance but instead train on more mask patterns.

wasted FLOPs, this inefficiency also incurs substantial memory and I/O overhead: the clean context occupies precious GPU memory during forward and backward passes, yet contributes zero gradients to the model update.

In this paper, we propose **Multi-Mask Training with context sharing** (**MMT**), a simple yet effective training paradigm that significantly improves computational utilization through a carefully designed attention mechanism. Our key insight is that instead of processing one mask pattern per sample, we can generate K different mask patterns for the same clean text and process them together in a single forward pass. As illustrated in Figure 2, by concatenating the shared clean prefix with K masked suffixes and employing a *context-sharing attention mask*, each masked segment attends to the clean context while remaining isolated from other segments. The resulting attention mask is highly sparse and can be efficiently implemented with FlexAttention (Dong et al., 2025). This design amortizes the cost of the clean context across K perturbations, improving the computational utilization from 50% to $\frac{K}{1+K}$. Crucially, we find that when using appropriate number of mask patterns K, **training on fewer samples with multiple perturbations per sample** yields comparable or even better model quality than training on more samples with single perturbations. Intuitively, by exposing the model to K different mask patterns of the same underlying text within a single forward pass, MMT acts as an implicit ensemble over perturbations, reducing the variance of the gradient signal and leading to more stable optimization. This *variance reduction* effect explains why MMT can achieve faster training without sacrificing or even improving model quality.

Beyond training, our context-sharing attention mask can also accelerate inference in diffusion-based reinforcement learning (RL). In diffusion RL, evaluating the probability of generated text requires Monte-Carlo sampling over multiple mask patterns, which is computationally expensive when performed naively. By batching these mask patterns together and applying our sparse attention mechanism, we can significantly reduce the cost of probability estimation during policy optimization.

Our contributions are summarized as follows:

- We identify the low computational utilization in Block Diffusion training, where the clean context incurs computation but does not contribute to the loss.
- We propose MMT, a multi-mask training paradigm with context sharing that processes multiple masked perturbations of the same sample in a single forward pass via a specially designed sparse attention pattern.
- Experiments show that MMT accelerates Block Diffusion training by $1.6\times$ with no loss in perplexity (TODO) or downstream task performance.

2 Related Work

2.1 Diffusion Language Models

Diffusion models have achieved remarkable success in continuous domains such as image and audio generation (Sohl-Dickstein et al., 2015; Ho et al., 2020; Song et al., 2021; Peebles and Xie, 2023). Recently, there has been growing interest in adapting diffusion-based methods to discrete text generation. Mercury (Khanna et al., 2025), Seed Diffusion (Song et al., 2025), and Gemini Diffusion (Google DeepMind, 2025) are industry-developed diffusion language models that demonstrate ultra-fast inference capabilities. In the open-source community, early methods (Li et al., 2022; Lovelace et al., 2023; Gong et al., 2023; Han et al., 2023; Strudel et al., 2022; Chen et al., 2023) applies continious space diffusion directly to language, while more recent methods (Hoogeboom et al., 2021, 2022; He et al., 2023; Lou et al., 2024; Ou et al., 2025; Sahoo et al., 2024) leverage discrete diffusion to achieve better scalability and performance. In this paper, we mainly focus on the discrete version, also known as **masked diffusion language models**. LLaDA (Nie et al., 2025b) trains an 8B-scale masked diffusion language model from scratch, demonstrating competitive performance with autoregressive counterparts. DREAM (Ye et al., 2025) adapts pre-trained autoregressive models into diffusion language models through continued training. These methods offer a promising alternative to autoregressive generation by enabling parallel token sampling, but they typically generate sequences of fixed length and lack the flexibility of variable-length generation.

2.2 Block Diffusion Language Models

Block Diffusion Language Models bridge the gap between autoregressive and diffusion paradigms by decomposing generation into sequential blocks. Ma et al. (2025); Wu et al. (2025b) use approximate KV caching to make a diffusion language model generate block-by-block. BD3-LMs (Arriola et al., 2025) trains a native block diffusion language model from scratch. Fast-DLLM v2 (Wu et al., 2025a) and SDAR (Cheng et al., 2025) adapt pre-trained autoregressive models into block diffusion models, enabling efficient conversion of existing AR checkpoints. LLaDA 2.0 (Bie et al., 2025) scales block diffusion language models up to 100B parameters, demonstrating the scalability of this approach. These models support variable-length generation, enable KV caching across blocks, and allow parallel token sampling within each block. However, their training suffers from low computational utilization due to the context duplication, which our method addresses.

2.3 Data Efficiency of Diffusion Language Models

Recent studies (Prabhudesai et al., 2025; Ni et al., 2025a,b) have revealed that diffusion language models are "super data learners" that can extract more information from the same amount of training data through diverse mask patterns. Another related approach is *complementary masking* (Wu et al., 2025a), which uses an inverted mask paired with a standard mask to enhance data utilization. However, these methods suffer from efficiency drawbacks. The former relies on implicit accumulation of diversity over repeated epochs with single-mask sampling, while the latter doubles the batch size and processes each perturbation independently, failing to exploit the shared clean context. In contrast, our method transforms this implicit benefit into an explicit, efficient training strategy. By processing multiple mask patterns simultaneously with **context sharing**, we actively enforce diverse supervision within each step while eliminating the computational waste of encoding the same context repeatedly.

3 Method

3.1 Preliminaries

Notation. Let $\mathbf{x} = (x_1, x_2, \dots, x_N)$ denote a sequence of N tokens, and let $\texttt{M}$ denote the special mask token. We denote the parameter of language model as θ. Notably, both autoregressive and diffusion-based language models share the same Transformer architecture, they differ only in their input representations and training objectives.

Autoregressive Language Models. Autoregressive (AR) language models is trained to minimize the negative log-likelihood of the sequence:

$$\mathcal{L}_{\text{AR}} = -\mathbb{E}_{\mathbf{x}\sim\mathcal{D}}\left[\sum_{i=1}^{N} \log p_\theta(x_i \mid x_{<i})\right]. \quad (1)$$

Masked Diffusion Language Models. Masked diffusion language models (MDLMs) (Nie et al., 2025b) define a discrete diffusion process over token sequences. Let $t \sim \mathcal{U}(0,1)$ denote a noise level, and let $\mathbf{x}^t$ denote the corrupted sequence obtained by independently masking each token x_i with probability t. Formally, for each position i:

$$x_i^t = \begin{cases} \texttt{M} & \text{with probability } t, \\ x_i & \text{otherwise.} \end{cases} \quad (2)$$

The model is trained to predict the original tokens from the corrupted sequence:

$$\begin{aligned}\mathcal{L}_{\text{MDLM}} &= -\mathbb{E}_{t,\mathbf{x},\mathbf{x}^t}\left[\frac{1}{t}\sum_{i=1}^{N}\mathbb{I}\left[x_i^t=\texttt{M}\right]\ell_{\theta,i}^{\text{MDLM}}(t)\right],\\ \ell_{\theta,i}^{\text{MDLM}}(t) &= \log p_\theta\left(x_i \,\middle|\, \mathbf{x}^t\right).\end{aligned} \tag{3}$$

Block Diffusion Language Models. Block diffusion language models (BDLMs) (Arriola et al., 2025) extend MDLMs by partitioning the sequence into contiguous blocks of size B. The sequence is divided into $N_b = N/B$ blocks, $\mathbf{x} = (\mathbf{b}_1, \ldots, \mathbf{b}_{N_b})$. During training, each block $r \in \{1, \ldots, N_b\}$ is assigned an independently sampled noise level $t_r \sim \mathcal{U}(0,1)$. Tokens within $\mathbf{b}_r$ are then masked independently with probability t_r, while the causal structure across blocks is maintained. For each token x_i, let $j = \lceil i/B \rceil$ denote its block index in the following objective:

$$\begin{aligned}\mathcal{L}_{\text{BDLM}} &= -\mathbb{E}_{\boldsymbol{t},\mathbf{x},\mathbf{x}^{\boldsymbol{t}}}\left[\sum_{i=1}^{N}\frac{\mathbb{I}\left[x_i^{\boldsymbol{t}}=\texttt{M}\right]}{t_j}\ell_{\theta,i}^{\text{BDLM}}(t_j)\right],\\ \ell_{\theta,i}^{\text{BDLM}}(t_j) &= \log p_\theta\left(x_i \,\middle|\, \mathbf{b}_{<j}, \mathbf{b}_j^{t_j}\right).\end{aligned} \tag{4}$$

where $\mathbf{b}_{<j}$ denotes the clean preceding blocks in $\mathbf{x}$, and $\mathbf{b}_j^t$ denotes the noisy version of the j-th block in $\mathbf{x}^t$. The specialized attention mask used for efficient BDLM training is illustrated in Figure 1.

3.2 MMT: Multi-Mask Training

Training formulation Given a clean sequence $\mathbf{x}$, MMT shares this clean sequence as the common context and generates K corrupted sequences for denoising training. For each view $k \in \{1, \ldots, K\}$, we independently sample block-wise noise levels $\boldsymbol{t}_k = (t_k^1, \ldots, t_k^{N_b})$, where $t_k^j \sim \mathcal{U}(0,1)$ is the noise level assigned to block j. The corresponding corrupted sequence $\mathbf{x}^{\boldsymbol{t}_k}$ is generated by masking each token x_i (where $j = \lceil i/B \rceil$) with probability t_k^j. The training objective averages the denoising losses over these K noised views:

$$\begin{aligned}\mathcal{L}_{\text{MMT}} &= -\mathbb{E}\left[\frac{1}{K}\sum_{k=1}^{K}\sum_{i=1}^{N}\frac{\mathbb{I}\left[x_i^{\boldsymbol{t}_k}=\texttt{M}\right]}{t_k^j}\ell_{\theta,i}^{\text{MMT}}(t_k^j)\right],\\ \ell_{\theta,i}^{\text{MMT}}(t_k^j) &= \log p_\theta\left(x_i \,\middle|\, \mathbf{b}_{<j}, \mathbf{b}_j^{\boldsymbol{t}_k}\right).\end{aligned} \tag{5}$$

where the expectation is taken over $\mathbf{x}$, the block-wise noise levels $\{\boldsymbol{t}_k\}_{k=1}^K$, and the corresponding corrupted sequences $\{\mathbf{x}^{\boldsymbol{t}_k}\}_{k=1}^K$. Here, j denotes the block index of token x_i, $\mathbf{b}_{<j}$ denotes the clean preceding blocks shared by all noised views, and $\mathbf{b}_j^{\boldsymbol{t}_k}$ denotes the noisy version of the j-th block in the k-th corrupted sequence.

Context-sharing attention mechanism To enable efficient parallel training of all K perturbations while maintaining correct token visibility, we construct an attention mask $\mathbf{M} \in \{0,1\}^{(K+1)N\times(K+1)N}$ over token indices. We use $k = 0$ for the clean sequence $\mathbf{x}$ and $k \in \{1, \ldots, K\}$ to index the K perturbations $\mathbf{x}^{\boldsymbol{t}_k}$. Let (k,i) denote the i-th token of the k-th sequence. The attention mask entry $M_{(k,i),(k',i')}$ determines whether token (k,i) can attend to token (k',i'):

$$M_{(k,i),(k',i')} = \begin{cases} 1, & \text{if } k'=0 \text{ and } \lceil i'/B \rceil < \lceil i/B \rceil,\\ 1, & \text{if } k'=k \text{ and } \lceil i'/B \rceil = \lceil i/B \rceil,\\ 0, & \text{otherwise.}\end{cases} \tag{6}$$

This mask ensures that each perturbation can attend to: (1) preceding blocks from the shared clean sequence, and (2) its own noisy block at the current position. The efficient training mask of MMT is illustrated in Figure 2.

Complementary masking via Context Sharing. We adopt the complementary masking strategy (Wu et al., 2025a). Instead of independently sampling K mask patterns, we sample $K/2$ pairs of complementary mask patterns. Crucially, unlike prior implementations that double the context computation cost, our context-sharing paradigm allows us to process complementary masks against a single clean context representation. The training objective of MMT becomes:

$$\mathcal{L}_{\text{MMT}} = -\mathbb{E}_{\{\boldsymbol{t}_v\}_{v=1}^{K/2},\mathbf{x},\{\mathbf{x}^{\boldsymbol{t}_v}\}_{v=1}^{K/2}}\left[\frac{1}{K}\sum_{v=1}^{K/2}\mathcal{L}_{\text{pair}}^{(v)}\right]. \tag{7}$$

Each pair of complementary masks consist an inverse mask patterns m and $\bar{m} = 1 - m$, generating two noisy sequences $\mathbf{x}^{\boldsymbol{t}_v}$ and $\bar{\mathbf{x}}^{\boldsymbol{t}_v}$, where

$$\bar{x}_i^{\boldsymbol{t}_v} = \begin{cases} x_i & \text{if } x_i^{\boldsymbol{t}_v} = \texttt{M},\\ \texttt{M} & \text{otherwise.}\end{cases} \tag{8}$$

Algorithm 1 MMT Training

Require: Model p_θ, dataset $\mathcal{D}$, number of mask pairs $K/2$, block size B

for each training iteration **do**
  Sample minibatch $\{\mathbf{x}\}$ from $\mathcal{D}$
  // Generate K/2 complementary mask pairs
  for $v = 1$ to $K/2$ **do**
    Sample block-wise noise $t_v \sim \text{Uniform}(0,1)^{N/B}$
    Sample $\mathbf{m}_v \in \{0,1\}^N$, with $m_{v,i} \sim \text{Bernoulli}(t_v^{\lceil i/B \rceil})$
    $\mathbf{x}^{t_v} \leftarrow$ Replace elements in $\mathbf{x}$ with $\mathtt{M}$ where $\mathbf{m}_v = 1$
    $\bar{\mathbf{x}}^{t_v} \leftarrow$ Replace elements in $\mathbf{x}$ with $\mathtt{M}$ where $\mathbf{m}_v = 0$
  end for
  // Construct input with shared clean context
  $\mathbf{X} \leftarrow \text{Concat}(\mathbf{x}, \mathbf{x}^{t_1}, \bar{\mathbf{x}}^{t_1}, \ldots, \mathbf{x}^{t_{K/2}}, \bar{\mathbf{x}}^{t_{K/2}})$
  Construct block-wise attention mask $\mathbf{M}$ using B per Eq. (6)
  // Forward and backward pass
  Compute denoising loss $\mathcal{L}_{\text{MMT}}$ on $\mathbf{X}$ with attention mask $\mathbf{M}$
  Update θ via gradient descent on $\mathcal{L}_{\text{MMT}}$
end for

The paired loss is then defined as

$$\mathcal{L}_{\text{pair}}^{(v)} = \sum_{i=1}^{N} [x_i^{t_v} = \mathtt{M}] \log p_\theta(x_i \mid \mathbf{b}_{<j}, \mathbf{b}_j^{t_v}) + \sum_{i=1}^{N} [\bar{x}_i^{t_v} = \mathtt{M}] \log p_\theta(x_i \mid \mathbf{b}_{<j}, \bar{\mathbf{b}}_j^{t_v}) \Bigg], \quad (9)$$

where $\bar{\mathbf{b}}_j^{t_v}$ is the j-th noisy block of $\bar{\mathbf{x}}^{t_i}$ (with $j = \lceil i/B \rceil$). We eliminate the normalization factor $1/t_v^j$ following Wu et al. (2025a), as the complementary masks jointly cover all N tokens.

Training algorithm. We summarize the complete training procedure of MMT in Algorithm 1.
(Working in Progress...)

3.3 Computational Complexity Analysis

We analyze the computational complexity of different language modeling paradigms. Let N denote the sequence length, P the total model parameters, L the number of transformer layers, and d the hidden dimension.

Per-token training cost. Following Chen et al. (2025), the FLOPs required to process one token with context length n is:

$$C_{\text{token}}(n) = \underbrace{2P}_{\text{non-attention}} + \underbrace{4nLd}_{\text{attention}}. \quad (10)$$

Here n is the number of visible tokens in the masked attention mechanism.

Autoregressive Transformers. For causal language models generating a sequence of length N, the total FLOPs is:

$$\begin{aligned} C_{\text{AR}} &= \sum_{t=1}^{N} C_{\text{token}}(t) = 2PN + \sum_{t=1}^{N} 4tLd \\ &= \underbrace{2PN}_{\text{non-attention}} + \underbrace{2N(N+1)Ld}_{\text{attention}}. \end{aligned} \quad (11)$$

Block Diffusion Language Models. Block diffusion models partition the sequence into $N_b = N/B$ blocks. At block j of the clean sequence $\mathbf{x}$, the model attends to all $(j-1)B$ preceding tokens plus the current block of size B. At block j of the noisy sequence $\mathbf{x}^t$, the model attends to all $(j-1)B$ preceding clean tokens plus the current noisy block of size B. That is, the j-th block of each sequence attends to jB tokens. The total FLOPs is

$$\begin{aligned} C_{\text{BDLM}} &= 2\sum_{i=1}^{N} C_{\text{token}}((\lceil i/B \rceil)B) \\ &= 2\sum_{j=1}^{N_b} B \cdot C_{\text{token}}(jB) \\ &= 2(2PN_bB + 2B^2N_b(N_b+1)Ld) \\ &= 2 \times \left(\underbrace{2PN}_{\text{non-attention}} + \underbrace{2N(N+B)Ld}_{\text{attention}} \right). \end{aligned} \quad (12)$$

MMT (Ours). Our context-sharing mechanism processes the clean context once and shares it across K perturbations. Similar to the above, we process $1+K$ sequences in total, where the j-th block of each sequence attends to jB tokens. The total FLOPs is

$$\begin{aligned} C_{\text{MMT}} &= (1+K)\sum_{j=1}^{N_b} B \cdot C_{\text{token}}(jB) \\ &= (1+K) \times \left(\underbrace{2PN}_{\text{non-attention}} + \underbrace{2N(N+B)Ld}_{\text{attention}} \right). \end{aligned} \quad (13)$$

The amortized cost per perturbation is:

$$\frac{C_{\text{MMT}}}{K} = (1+1/K)\times\left(\underbrace{2PN}_{\text{non-attention}} + \underbrace{2N(N+B)Ld}_{\text{attention}}\right). \tag{14}$$

In typical training settings, the sequence length N is at least 2048 or 4096, while the block size B is usually set to small values such as 32 or 64 (Wu et al., 2025a). Since $B \ll N$, we have $N(N+B) \approx N^2$ and $N(N+1) \approx N^2$. Table 1 summarizes the average training FLOPs per noise perturbation under this approximation. As shown, MMT significantly improves the FLOPs efficiency compared to block diffusion, which can be viewed as a special case of our method with $K = 1$.

Table 1: Average FLOPs per sequence/perturbation. For MMT, the cost is amortized over K perturbations, and $(1+1/K) \approx 1$ when K is reasonably large.

	AR	BDLM	MMT (Ours)
Non-attention	$2PN$	$4PN$	$(1+1/K)\cdot 2PN$
Attention	$2N^2Ld$	$4N^2Ld$	$(1+1/K)\cdot 2N^2Ld$

-

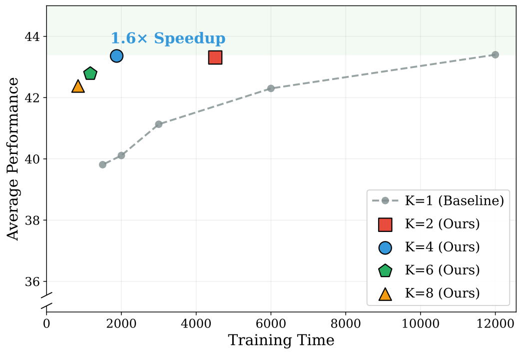

Figure 3: Training efficiency: Time vs. Performance. The gray dashed line shows K=1 baseline performance at different training steps. Our method (colored markers) achieves better performance with significantly less training time.

TODO: Gradient Norm Analysis Figure

Figure 4: Gradient norm per step with different K values. Larger K leads to lower gradient norm due to gradient averaging across multiple mask patterns.

4 Experiment

We conduct experiments under two distinct training paradigms, "AR to Diffusion" and "Training Diffusion from Scratch", to comprehensively evaluate our proposed method.

4.1 AR to Diffusion

For **AR to Diffusion** experiments, we adopt the experimental settings and framework from Fast-dLLM v2 (Wu et al., 2025a) and use it as our baseline to evaluate the performance gains achieved by our proposed improvements.

Models We initialize our model from Qwen2.5-1.5B-Instruct (**?**) and fine-tune it into a block diffusion language model following the training recipe of Fast-dLLM v2 (Wu et al., 2025a). The fine-tuning is performed on approximately 3B tokens sampled from the LLaMA-Nemotron post-training dataset (Bercovich et al., 2025). The block size B is set to 32, the batch size is set to 256 and the context length is set to 2048. The learning rate is set to $2e-5$.

Benchmarks We conduct evaluation on MMLU (Hendrycks et al., 2021a) and GPQA (Rein et al., 2024) benchmarks for general ability evaluation. We conduct evaluation on GSM8K (Cobbe et al., 2021), MATH (Hendrycks et al., 2021b), MBPP (Austin et al., 2021), and HumanEval (Chen et al., 2021) benchmarks for mathematical reasoning and code generation tasks. All benchmarks are evaluated in the zero-shot setting except that MMLU is evaluated in the 5-shot setting.

Results

Ablations

Gradient Analysis We analyze the gradient norm per step for different K settings to understand the optimization dynamics of our method. As shown in Figure 4, we observe that larger K values lead to lower gradient norms per step. This is because when multiple mask patterns share the same clean context, the gradients from different masked views are aggregated and averaged, resulting in a more stable and smoothed gradient direction. The reduced gradient variance per step allows for more consistent optimization, which partially explains why our method maintains competitive performance even with fewer training steps.

Table 2: Comparison of different K settings of MMT in AR to Diffusion setting.

Method	GSM8K	MATH	MMLU	GPQA	IFEval	MBPP	HumanEval	Avg.
K=1, 12000 steps	$64.54_{\pm1.48}$	$29.74_{\pm0.14}$	$49.51_{\pm1.48}$	$26.27_{\pm0.91}$	$41.47_{\pm1.07}$	$46.95_{\pm2.87}$	$45.32_{\pm1.54}$	$43.40_{\pm0.43}$
K=2, 6000 steps	$63.43_{\pm1.75}$	$30.96_{\pm0.19}$	$50.73_{\pm0.77}$	$26.49_{\pm0.46}$	$41.03_{\pm1.39}$	$50.06_{\pm1.25}$	$40.45_{\pm3.46}$	$43.31_{\pm0.25}$
K=1, 6000 steps	$61.99_{\pm0.58}$	$28.29_{\pm0.68}$	$51.17_{\pm0.51}$	$26.56_{\pm0.00}$	$42.70_{\pm1.82}$	$46.95_{\pm0.23}$	$38.41_{\pm2.20}$	$42.30_{\pm0.25}$
K=4, 3000 steps	$62.25_{\pm0.89}$	$29.32_{\pm0.47}$	$51.60_{\pm0.25}$	$26.86_{\pm0.52}$	$42.70_{\pm1.30}$	$48.12_{\pm0.60}$	$42.68_{\pm1.61}$	$43.36_{\pm0.13}$
K=1, 3000 steps	$59.21_{\pm0.72}$	$25.85_{\pm0.74}$	$52.39_{\pm0.22}$	$26.41_{\pm0.68}$	$41.90_{\pm0.93}$	$46.95_{\pm2.38}$	$35.17_{\pm1.96}$	$41.13_{\pm0.07}$
K=6, 2000 steps	$62.37_{\pm0.37}$	$28.01_{\pm0.64}$	$52.24_{\pm0.29}$	$27.09_{\pm0.65}$	$40.73_{\pm1.76}$	$48.12_{\pm0.81}$	$40.85_{\pm2.11}$	$42.78_{\pm0.29}$
K=1, 2000 steps	$58.45_{\pm0.08}$	$24.76_{\pm0.58}$	$52.43_{\pm0.22}$	$26.56_{\pm0.23}$	$41.16_{\pm2.62}$	$43.45_{\pm3.14}$	$33.95_{\pm1.27}$	$40.11_{\pm0.75}$
K=8, 1500 steps	$61.46_{\pm1.03}$	$27.83_{\pm0.69}$	$52.56_{\pm0.38}$	$26.86_{\pm0.68}$	$41.71_{\pm3.62}$	$47.60_{\pm0.81}$	$38.62_{\pm2.14}$	$42.38_{\pm0.45}$
K=1, 1500 steps	$56.03_{\pm0.35}$	$23.97_{\pm0.41}$	$52.74_{\pm0.13}$	$27.38_{\pm0.13}$	$39.99_{\pm2.78}$	$43.58_{\pm1.17}$	$34.96_{\pm2.14}$	$39.81_{\pm0.74}$

Table 3: Ablation study of complementary masking of MMT in AR to Diffusion setting.

Method	Complementary	GSM8K	MATH	MMLU	GPQA	IFEval	MBPP	HumanEval	Avg.
K=2	✓	$63.43_{\pm1.75}$	$30.96_{\pm0.19}$	$50.73_{\pm0.77}$	$26.49_{\pm0.46}$	$41.03_{\pm1.39}$	$50.06_{\pm1.25}$	$40.45_{\pm3.46}$	$43.31_{\pm0.25}$
K=2	✗	$63.36_{\pm1.56}$	$30.10_{\pm0.30}$	$50.91_{\pm0.15}$	$26.41_{\pm0.78}$	$42.82_{\pm1.08}$	$49.42_{\pm1.78}$	$43.29_{\pm2.20}$	$43.76_{\pm0.10}$
K=4	✓	$62.25_{\pm0.89}$	$29.32_{\pm0.47}$	$51.60_{\pm0.25}$	$26.86_{\pm0.52}$	$42.70_{\pm1.30}$	$48.12_{\pm0.60}$	$42.68_{\pm1.61}$	$43.36_{\pm0.13}$
K=4	✗	$61.58_{\pm1.67}$	$29.11_{\pm0.52}$	$52.21_{\pm0.23}$	$26.12_{\pm1.02}$	$43.90_{\pm1.63}$	$50.59_{\pm1.59}$	$41.16_{\pm2.71}$	$43.52_{\pm0.67}$

4.2 Training Diffusion from Scratch

For **Training Diffusion from Scratch**, we follow the experimental settings and framework established by SMDM (Nie et al., 2025a), using it as our baseline to demonstrate the effectiveness of our approach.

Models We train masked diffusion language models from scratch at three different scales: 170M, 336M, and 1028M parameters. All models are trained on the SlimPajama (Soboleva et al., 2023) and StarCoder (Li et al., 2023) datasets. We use a batch size of 256 and a context length of 2048 tokens. The learning rate is set to $2e-4$ and the learning rate warmup is set to 1% of the total training steps.

Benchmarks Following SMDM (Nie et al., 2025a), we use eight benchmarks covering commonsense reasoning and reading comprehension tasks, including ARC-e (Clark et al., 2018), BoolQ (Clark et al., 2019), Hellaswag (Zellers et al., 2019), Obqa (Mihaylov et al., 2018), PIQA (Bisk et al., 2020), RACE (Lai et al., 2017), SIQA (Sap et al., 2019), LAMBADA (Paperno et al., 2016). All benchmarks are evaluated in the zero-shot setting.

4.3 Speedup Analysis

We analyze the computational speedup of our context-sharing attention mechanism at both the attention kernel level and the end-to-end training level. To ensure a fair comparison, we fix the batch size to 4 and the sequence length to $N = 2048$. The baseline approach processes K mask patterns independently, resulting in a batch size of $4K$ with sequence length $2N$. In contrast, our method maintains the same batch size of 4 but increases the sequence length to $(1+K)N$ by concatenating the shared clean text with K masked texts.

Attention Kernel Speedup Our context-sharing attention mask is highly sparse: each masked segment only attends to the shared clean prefix and itself, while remaining isolated from other segments. By leveraging FlexAttention (Dong et al., 2025), we exploit this sparsity pattern to achieve significant speedup in the attention computation. As shown in Figure 5, our sparse attention kernel achieves up to $TODO\times$ speedup compared to dense attention when the number of mask patterns K increases.

End-to-End Training Speedup We measure the end-to-end training speedup by comparing the wall-clock time per training step of our method against the baseline. As shown in Figure 6, our method achieves $TODO\times$ end-to-end training speedup

TODO: Attention Speedup Figure

Figure 5: Attention kernel speedup with different number of mask patterns K.

TODO: End-to-End Speedup Figure

Figure 6: End-to-end training speedup with different number of mask patterns K.

with $K = 4$ mask patterns, demonstrating the practical efficiency gains of our approach.

5 Conclusion

Limitations

References

Marianne Arriola, Aaron Gokaslan, Justin T Chiu, Zhihan Yang, Zhixuan Qi, Jiaqi Han, Subham Sekhar Sahoo, and Volodymyr Kuleshov. 2025. Block diffusion: Interpolating between autoregressive and diffusion language models. *arXiv preprint arXiv:2503.09573*.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and 1 others. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*.

Akhiad Bercovich, Itay Levy, Izik Golan, Mohammad Dabbah, Ran El-Yaniv, Omri Puny, Ido Galil, Zach Moshe, Tomer Ronen, Najeeb Nabwani, Ido Shahaf, Oren Tropp, Ehud Karpas, Ran Zilberstein, Jiaqi Zeng, Soumye Singhal, Alexander Bukharin, Yian Zhang, Tugrul Konuk, and 114 others. 2025. Llama-nemotron: Efficient reasoning models. *Preprint*, arXiv:2505.00949.

Tiwei Bie, Maosong Cao, Kun Chen, Lun Du, Mingliang Gong, Zhuochen Gong, Yanmei Gu, Jiaqi Hu, Zenan Huang, Zhenzhong Lan, Chengxi Li, Chongxuan Li, Jianguo Li, Zehuan Li, Huabin Liu, Ling Liu, Guoshan Lu, Xiaocheng Lu, Yuxin Ma, and 12 others. 2025. Llada 2.0: Scaling up diffusion language models to 100b. *Preprint*, arXiv:2512.15745.

Yonatan Bisk, Rowan Zellers, Jianfeng Gao, Yejin Choi, and 1 others. 2020. Piqa: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pages 7432–7439.

Rishi Bommasani, Drew A Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, and 1 others. 2021. On the opportunities and risks of foundation models. *Preprint*.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, and 1 others. 2020. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, and 1 others. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.

Ting Chen, Ruixiang Zhang, and Geoffrey Hinton. 2023. Analog bits: Generating discrete data using diffusion models with self-conditioning. In *The Eleventh International Conference on Learning Representations*.

Yingfa Chen, Yutong Wu, Chenyang Song, Zhen Leng Thai, Xingyu Shen, Xu Han, Zhiyuan Liu, and Maosong Sun. 2025. Cost-optimal grouped-query attention for long-context modeling. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 5360–5376.

Shuang Cheng, Yihan Bian, Dawei Liu, Linfeng Zhang, Qian Yao, Zhongbo Tian, Wenhai Wang, Qipeng Guo, Kai Chen, Biqing Qi, and 1 others. 2025. Sdar: A synergistic diffusion-autoregression paradigm for scalable sequence generation. *arXiv preprint arXiv:2510.06303*.

Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. 2019. Boolq: Exploring the surprising difficulty of natural yes/no questions. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 2924–2936.

Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.

Juechu Dong, Boyuan Feng, Driss Guessous, Yanbo Liang, and Horace He. 2025. Flexattention: A programming model for generating fused attention variants. *Proceedings of Machine Learning and Systems*, 7.

Shansan Gong, Mukai Li, Jiangtao Feng, Zhiyong Wu, and Lingpeng Kong. 2023. Diffuseq: Sequence to sequence text generation with diffusion models. In *The Eleventh International Conference on Learning Representations*.

Google DeepMind. 2025. Gemini diffusion. https://blog.google/technology/google-deepmind/gemini-diffusion/. Accessed: 2026-05-11.

Xiaochuang Han, Sachin Kumar, and Yulia Tsvetkov. 2023. Ssd-lm: Semi-autoregressive simplex-based diffusion language model for text generation and modular control. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 11575–11596.

Xu Han, Zhengyan Zhang, Ning Ding, Yuxian Gu, Xiao Liu, Yuqi Huo, Jiezhong Qiu, Yuan Yao, Ao Zhang, Liang Zhang, and 1 others. 2021. Pre-trained models: Past, present and future. *AI Open*, 2:225–250.

Zhengfu He, Tianxiang Sun, Qiong Tang, Kuanning Wang, Xuan-Jing Huang, and Xipeng Qiu. 2023. Diffusionbert: Improving generative masked language models with diffusion models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 4521–4534.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2021a. Measuring massive multitask language understanding. In *International Conference on Learning Representations*.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021b. Measuring mathematical problem solving with the math dataset. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*.

Jonathan Ho, Ajay Jain, and Pieter Abbeel. 2020. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851.

Emiel Hoogeboom, Alexey A Gritsenko, Jasmijn Bastings, Ben Poole, Rianne van den Berg, and Tim Salimans. 2022. Autoregressive diffusion models. In *International Conference on Learning Representations*.

Emiel Hoogeboom, Didrik Nielsen, Priyank Jaini, Patrick Forré, and Max Welling. 2021. Argmax flows and multinomial diffusion: Learning categorical distributions. *Advances in neural information processing systems*, 34:12454–12465.

Samar Khanna, Siddhant Kharbanda, Shufan Li, Harshit Varma, Eric Wang, Sawyer Birnbaum, Ziyang Luo, Yanis Miraoui, Akash Palrecha, Stefano Ermon, and 1 others. 2025. Mercury: Ultra-fast language models based on diffusion. *arXiv preprint arXiv:2506.17298*, 1.

Guokun Lai, Qizhe Xie, Hanxiao Liu, Yiming Yang, and Eduard Hovy. 2017. Race: Large-scale reading comprehension dataset from examinations. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, pages 785–794.

Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, and 1 others. 2023. Starcoder: may the source be with you! *arXiv preprint arXiv:2305.06161*.

Xiang Li, John Thickstun, Ishaan Gulrajani, Percy S Liang, and Tatsunori B Hashimoto. 2022. Diffusion-lm improves controllable text generation. *Advances in neural information processing systems*, 35:4328–4343.

Aaron Lou, Chenlin Meng, and Stefano Ermon. 2024. Discrete diffusion modeling by estimating the ratios of the data distribution. In *Forty-first International Conference on Machine Learning*.

Justin Lovelace, Varsha Kishore, Chao Wan, Eliot Shekhtman, and Kilian Q Weinberger. 2023. Latent diffusion for language generation. *Advances in Neural Information Processing Systems*, 36:56998–57025.

Xinyin Ma, Runpeng Yu, Gongfan Fang, and Xinchao Wang. 2025. dkv-cache: The cache for diffusion language models. *arXiv preprint arXiv:2505.15781*.

Todor Mihaylov, Peter Clark, Tushar Khot, and Ashish Sabharwal. 2018. Can a suit of armor conduct electricity? a new dataset for open book question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2381–2391.

Jinjie Ni, Qian Liu, Longxu Dou, Chao Du, Zili Wang, Hang Yan, Tianyu Pang, and Michael Qizhe Shieh. 2025a. Diffusion language models are super data learners. *arXiv preprint arXiv:2511.03276*.

Jinjie Ni, Qian Liu, Chao Du, Longxu Dou, Hang Yan, Zili Wang, Tianyu Pang, and Michael Qizhe Shieh. 2025b. Training optimal large diffusion language models. *arXiv preprint arXiv:2510.03280*.

Shen Nie, Fengqi Zhu, Chao Du, Tianyu Pang, Qian Liu, Guangtao Zeng, Min Lin, and Chongxuan Li. 2025a. Scaling up masked diffusion models on text. In *The Thirteenth International Conference on Learning Representations*.

Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. 2025b. Large language diffusion models. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.

OpenAI. 2023. GPT-4 technical report. *Preprint*.

Jingyang Ou, Shen Nie, Kaiwen Xue, Fengqi Zhu, Jiacheng Sun, Zhenguo Li, and Chongxuan Li. 2025. Your absorbing discrete diffusion secretly models the conditional distributions of clean data. In *The Thirteenth International Conference on Learning Representations*.

Denis Paperno, Germán Kruszewski, Angeliki Lazaridou, Ngoc-Quan Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Raquel Fernández. 2016. The LAMBADA dataset: Word prediction requiring a broad discourse context. In *Proceedings of the 54th annual meeting of the association for computational linguistics (volume 1: Long papers)*, pages 1525–1534.

William Peebles and Saining Xie. 2023. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 4195–4205.

Mihir Prabhudesai, Mengning Wu, Amir Zadeh, Katerina Fragkiadaki, and Deepak Pathak. 2025. Diffusion beats autoregressive in data-constrained settings. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.

David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R Bowman. 2024. Gpqa: A graduate-level google-proof q&a benchmark. In *First Conference on Language Modeling*.

Subham Sahoo, Marianne Arriola, Yair Schiff, Aaron Gokaslan, Edgar Marroquin, Justin Chiu, Alexander Rush, and Volodymyr Kuleshov. 2024. Simple and effective masked diffusion language models. *Advances in Neural Information Processing Systems*, 37:130136–130184.

Maarten Sap, Hannah Rashkin, Derek Chen, Ronan Le Bras, and Yejin Choi. 2019. Social iqa: Commonsense reasoning about social interactions. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 4463–4473.

Daria Soboleva, Faisal Al-Khateeb, Robert Myers, Jacob R Steeves, Joel Hestness, and Nolan Dey. 2023. Slimpajama: A 627b token, cleaned and deduplicated version of redpajama. *Cerebras blog post*.

Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. 2015. Deep unsupervised learning using nonequilibrium thermodynamics. In *International conference on machine learning*, pages 2256–2265. pmlr.

Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. 2021. Score-based generative modeling through stochastic differential equations. In *International Conference on Learning Representations*.

Yuxuan Song, Zheng Zhang, Cheng Luo, Pengyang Gao, Fan Xia, Hao Luo, Zheng Li, Yuehang Yang, Hongli Yu, Xingwei Qu, and 1 others. 2025. Seed diffusion: A large-scale diffusion language model with high-speed inference. *arXiv preprint arXiv:2508.02193*.

Robin Strudel, Corentin Tallec, Florent Altché, Yilun Du, Yaroslav Ganin, Arthur Mensch, Will Grathwohl, Nikolay Savinov, Sander Dieleman, Laurent Sifre, and 1 others. 2022. Self-conditioned embedding diffusion for text generation. *arXiv preprint arXiv:2211.04236*.

Yuchuan Tian, Yuchen Liang, Jiacheng Sun, Shuo Zhang, Guangwen Yang, Yingte Shu, Sibo Fang, Tianyu Guo, Kai Han, Chao Xu, and 1 others. 2025. From next-token to next-block: A principled adaptation path for diffusion llms. *arXiv preprint arXiv:2512.06776*.

Chengyue Wu, Hao Zhang, Shuchen Xue, Shizhe Diao, Yonggan Fu, Zhijian Liu, Pavlo Molchanov, Ping Luo, Song Han, and Enze Xie. 2025a. Fast-dllm v2: Efficient block-diffusion llm. *arXiv preprint arXiv:2509.26328*.

Chengyue Wu, Hao Zhang, Shuchen Xue, Zhijian Liu, Shizhe Diao, Ligeng Zhu, Ping Luo, Song Han, and Enze Xie. 2025b. Fast-dllm: Training-free acceleration of diffusion llm by enabling kv cache and parallel decoding. *arXiv preprint arXiv:2505.22618*.

Jiacheng Ye, Zhihui Xie, Lin Zheng, Jiahui Gao, Zirui Wu, Xin Jiang, Zhenguo Li, and Lingpeng Kong. 2025. Dream 7b: Diffusion large language models. *arXiv preprint arXiv:2508.15487*.

Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. 2019. Hellaswag: Can a machine really finish your sentence? *arXiv preprint arXiv:1905.07830*.

A Theoretical Analysis

A.1 Unbiasedness of MMT Loss Estimator

We prove that the MMT training objective is an unbiased estimator of the BDLM objective. Recall that the BDLM loss for a single sample $\mathbf{x}$ is:

$$\mathcal{L}_{\text{BDLM}}(\mathbf{x}) = -\mathbb{E}_{t,\mathbf{x}^t}\left[\frac{1}{t}\sum_{i=1}^{N}\mathbb{1}[x_i^t = \mathtt{M}]\log p_\theta(x_i \mid \mathbf{b}_{<block(i)}, \mathbf{b}^t_{bloc} \tag{15}$$

where the expectation is over the noise level $t \sim \mathcal{U}(0,1)$ and the stochastic masking process.

Theorem A.1 (Unbiasedness). *Let $\{t_k\}_{k=1}^K$ be K i.i.d. samples from $\mathcal{U}(0,1)$, and let $\{\mathbf{x}^{t_k}\}_{k=1}^K$ be the corresponding independently corrupted sequences. Then:*

$$-\mathbb{E}_{\{t_k\},\{\mathbf{x}^{t_k}\}}\left[\frac{1}{K}\sum_{k=1}^{K}\frac{1}{t_k}\sum_{i=1}^{N}\mathbb{1}[x_i^{t_k} = \mathtt{M}]\log p_\theta(x_i \mid \mathbf{b}_{<block(i)}, \mathbf{b}^{t_k}_{blo} \tag{16}$$

Proof. Define the per-perturbation loss:

$$\ell_k(\mathbf{x}) \triangleq -\frac{1}{t_k}\sum_{i=1}^{N} \mathbb{1}[x_i^{t_k} = \mathtt{M}] \log p_\theta(x_i \mid \mathbf{b}_{<block(i)}, \mathbf{b}_{block(i)}^{t_k}). \tag{17}$$

By linearity of expectation:

$$\mathbb{E}\left[\frac{1}{K}\sum_{k=1}^{K}\ell_k(\mathbf{x})\right] = \frac{1}{K}\sum_{k=1}^{K}\mathbb{E}[\ell_k(\mathbf{x})]. \tag{18}$$

Since each $(t_k, \mathbf{x}^{t_k})$ is sampled from the same distribution as $(t, \mathbf{x}^t)$ in BDLM:

$$\mathbb{E}[\ell_k(\mathbf{x})] = \mathbb{E}_{t,\mathbf{x}^t}[\ell(\mathbf{x})] = \mathcal{L}_{\text{BDLM}}(\mathbf{x}), \quad \forall k \in \{1, \ldots, K\}. \tag{19}$$

Therefore:

$$\mathbb{E}\left[\frac{1}{K}\sum_{k=1}^{K}\ell_k(\mathbf{x})\right] = \frac{1}{K}\cdot K \cdot \mathcal{L}_{\text{BDLM}}(\mathbf{x}) = \mathcal{L}_{\text{BDLM}}(\mathbf{x}). \tag{20}$$

□

Remark A.2. The key observation is that MMT does not introduce any bias—it simply uses K Monte Carlo samples to estimate the same expected loss as BDLM. The samples share the same clean context $\mathbf{x}$, but each mask pattern is sampled independently, preserving the unbiasedness property.

However, this comparison is misleading because BDLM requires K forward passes while MMT only requires 1 forward pass (with efficient context sharing). When computational cost is properly accounted for, MMT achieves better variance-to-compute ratio by reducing mask noise through averaging, while maintaining training efficiency via context reuse.